

Function Composition

[*Person, Char, B, Record, ValidatedRecord, ProcessedRecord, Output*]

Example 1 : Basic Forward Composition

Forward composition applies f first, then g:

$$\begin{array}{l|l} f : \mathbb{N} \rightarrow \mathbb{N} \\ g : \mathbb{N} \rightarrow \mathbb{N} \\ h : \mathbb{N} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{N} \bullet f(n) = n + 1 \\ \forall n : \mathbb{N} \bullet g(n) = n * 2 \\ h = f \circ g \end{array}$$

$$h(n) = g(f(n)) = g(n + 1) = (n + 1) * 2 = 2n + 2$$

Example 2 : Backward Composition

Backward composition applies the second function first (using comp):

$$\begin{array}{l|l} f2 : \mathbb{N} \rightarrow \mathbb{N} \\ g2 : \mathbb{N} \rightarrow \mathbb{N} \\ h2 : \mathbb{N} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{N} \bullet f2(n) = n + 1 \\ \forall n : \mathbb{N} \bullet g2(n) = n * 2 \\ h2 = f2 \circ g2 \end{array}$$

$$h2(n) = f2(g2(n)) = f2(n * 2) = (n * 2) + 1 = 2n + 1$$

Example 3 : Comparison - Forward vs Backward

Forward composition f ∘ g: Input → f → g → Output (f ∘ g)(x) = g(f(x))

Backward composition f o g: Input → g → f → Output (f o g)(x) = f(g(x))

Notice the reversal: ∘ applies left-to-right, o applies right-to-left.

Example 4 : Three - Function Composition

$$\begin{array}{l|l} addOne : \mathbb{N} \rightarrow \mathbb{N} \\ double : \mathbb{N} \rightarrow \mathbb{N} \\ square : \mathbb{N} \rightarrow \mathbb{N} \\ pipeline : \mathbb{N} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{N} \bullet addOne(n) = n + 1 \\ \forall n : \mathbb{N} \bullet double(n) = 2 * n \\ \forall n : \mathbb{N} \bullet square(n) = n * n \\ pipeline = addOne \circ double \circ square \end{array}$$

$$pipeline(n) = square(double(addOne(n))) = square(double(n + 1)) = square(2(n + 1)) = (2(n + 1))^2$$

Example 5 : Identity Composition Laws

$identity : \mathbb{N} \rightarrow \mathbb{N}$ $anyFunc : \mathbb{N} \rightarrow \mathbb{N}$ $leftId : \mathbb{N} \rightarrow \mathbb{N}$ $rightId : \mathbb{N} \rightarrow \mathbb{N}$	$\forall n : \mathbb{N} \bullet identity(n) = n$ $\forall n : \mathbb{N} \bullet anyFunc(n) = n * n$ $leftId = identity \circ anyFunc$ $rightId = anyFunc \circ identity$
--	--

$leftId = anyFunc$ (identity is left identity for $\circ$) $rightId = anyFunc$ (identity is right identity for $\circ$)

Example 6 : Associativity

Composition is associative:

$$(f \circ g) \circ h = f \circ (g \circ h)$$

You can group compositions in any order—the result is the same.

$f3 : \mathbb{N} \rightarrow \mathbb{N}$ $g3 : \mathbb{N} \rightarrow \mathbb{N}$ $h3 : \mathbb{N} \rightarrow \mathbb{N}$ $comp1 : \mathbb{N} \rightarrow \mathbb{N}$ $comp2 : \mathbb{N} \rightarrow \mathbb{N}$	$\forall n : \mathbb{N} \bullet f3(n) = n + 1$ $\forall n : \mathbb{N} \bullet g3(n) = n * 2$ $\forall n : \mathbb{N} \bullet h3(n) = n * n$ $comp1 = (f3 \circ g3) \circ h3$ $comp2 = f3 \circ (g3 \circ h3)$
--	--

$comp1 = comp2$ for all inputs.

Example 7 : Composition with Partial Functions

$safeSqrt : \mathbb{N} \rightarrow \mathbb{N}$ $addTen : \mathbb{N} \rightarrow \mathbb{N}$ $composed : \mathbb{N} \rightarrow \mathbb{N}$	$\forall n : \mathbb{N} \bullet n * n = n \Rightarrow safeSqrt(n) = n$ $\forall n : \mathbb{N} \bullet addTen(n) = n + 10$ $composed = safeSqrt \circ addTen$
--	---

$composed$ is only defined where $safeSqrt$ is defined, then adds 10 to the result.

Example 8 : Relation Composition

Composition works for relations too:

$parent : Person \leftrightarrow Person$ $grandparent : Person \leftrightarrow Person$	$grandparent = parent \circ parent$
---	-------------------------------------

$grandparent$ relates a person to their grandparents via two parent links.

Example 9 : Function Pipeline Example

$toLowerCase : seq\ Char \rightarrow seq\ Char$ $trim : seq\ Char \rightarrow seq\ Char$ $reverse : seq\ Char \rightarrow seq\ Char$ $process : seq\ Char \rightarrow seq\ Char$	$process = toLowerCase \ ; \ trim \ ; \ reverse$
---	--

Process a string by converting to lowercase, trimming whitespace, then reversing.

Example 10 : Mathematical Functions

$squareFunc : \mathbb{N} \rightarrow \mathbb{N}$ $tripleFunc : \mathbb{N} \rightarrow \mathbb{N}$ $combined : \mathbb{N} \rightarrow \mathbb{N}$	$\forall n : \mathbb{N} \bullet squareFunc(n) = n * n$ $\forall n : \mathbb{N} \bullet tripleFunc(n) = 3 * n$ $combined = squareFunc \ ; \ tripleFunc$
--	--

Combined applies squareFunc first, then triples the result.

Example 11 : Inverse via Composition

$encrypt : seq\ Char \rightarrow seq\ Char$ $decrypt : seq\ Char \rightarrow seq\ Char$ $identity_check : seq\ Char \rightarrow seq\ Char$	$identity_check = encrypt \ ; \ decrypt$
---	---

If encrypt and decrypt are inverses, then identity_check should be the identity function.

Example 12 : Partial Composition

When composing $f : A \multimap B$ and $g : C \multimap D$, we need $\text{ran } f \subseteq \text{dom } g$.

$f4 : \mathbb{N} \multimap \mathbb{N}$ $g4 : \mathbb{N} \multimap \mathbb{N}$ $h4 : \mathbb{N} \multimap \mathbb{N}$	$f4 = \{1 \mapsto 10, 2 \mapsto 20\}$ $g4 = \{10 \mapsto 100, 20 \mapsto 200\}$ $h4 = f4 \ ; \ g4$
--	--

$h4 = \{1 \mapsto 100, 2 \mapsto 200\}$. Composition is defined where ranges align.

Example 13 : Monoid Structure

Functions from $A \rightarrow A$ with composition form a monoid: 1. Composition is associative: $(f \ ; \ g) \ ; \ h = f \ ; \ (g \ ; \ h)$ 2. Identity element: $\text{id } x = x$, $\text{id} \ ; \ f = f = f \ ; \ \text{id}$ 3. Closure: composing $A \rightarrow A$ functions gives another $A \rightarrow A$ function

Example 14 : Practical Example - Data Processing Pipeline

In practice, you might compose functions to create data processing pipelines: $validate \ ; \ process \ ; \ format$. Each stage transforms data through a series of operations. The notation $f \ ; \ g \ ; \ h$ means "apply f, then g, then h" which creates a pipeline from input to output.

Example 15 : Composition Operator Precedence

Composition operators bind tighter than logical operators:

$f \circ g = h$ means $(f \circ g) = h$, not $f \circ (g = h)$

Use parentheses to clarify: $(f \circ g)$ or $f \circ (g \circ h)$

Example 16 : Self - Composition

$$\begin{array}{l} \text{doubleIt} : \mathbb{N} \rightarrow \mathbb{N} \\ \text{quadrupleIt} : \mathbb{N} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{N} \bullet \text{doubleIt}(n) = 2 * n \\ \text{quadrupleIt} = \text{doubleIt} \circ \text{doubleIt} \end{array}$$

$$\text{quadrupleIt}(n) = \text{doubleIt}(\text{doubleIt}(n)) = 2 * (2 * n) = 4n$$

Example 17 : Iterated Composition

$$\begin{array}{l} \text{increment} : \mathbb{N} \rightarrow \mathbb{N} \\ \text{addFive} : \mathbb{N} \rightarrow \mathbb{N} \\ \hline \forall n : \mathbb{N} \bullet \text{increment}(n) = n + 1 \\ \text{addFive} = \text{increment} \circ \text{increment} \circ \text{increment} \circ \text{increment} \circ \text{increment} \end{array}$$

Composing increment 5 times gives addFive.

Example 18 : Functional Programming Style

Composition enables a functional programming style:

Instead of: $\text{result} = h(g(f(x)))$ Use: $\text{pipeline} = f \circ g \circ h$; $\text{result} = \text{pipeline}(x)$

This separates pipeline definition from application, improving modularity.

Example 19 : Best Practices

When using composition: 1. Choose forward (o9) or backward (o) consistently in your codebase 2. Document the data flow direction 3. Break complex pipelines into named intermediate steps 4. Type-check that domains and ranges align 5. Use composition to create reusable pipelines 6. Test individual functions before composing them 7. Remember: $\circ$ reads left-to-right (natural for pipelines), $\bullet$ reads right-to-left (traditional math notation)